САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №7
«Облачное развёртывание веб-приложения»

Выполнил:

Якшин Артемий

Группа БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Настроить облачное развёртывание и оркестрацию микросервисного веб-приложения в облачной инфраструктуре (Kubernetes / Managed Cloud Services) по техническому дизайну, разработанному в ДЗ7.

Ход работы

1. Что разворачивается

По ДЗ7 реализуется вариант А: Kubernetes-кластер k3s, запущенный через k3d в контейнере на облачной ВМ. Внутри кластера работают шесть stateless-сервисов приложения, PostgreSQL и RabbitMQ (StatefulSet с постоянными томами), Ingress-контроллер Traefik, сетевые политики и ежедневный бэкап БД.

Компонент	**Версия / параметры**
k3d	v5.9.0 (sha256 бинарника закреплён в Ansible-роли)
k3s (Kubernetes)	v1.36.6+k3s1, образ `rancher/k3s:v1.36.6-k3s1`, containerd 2.3.7
kubectl	v1.36.6 (sha256 проверяется по официальному `.sha256`)
Узел	`k3d-rb-server-0`: `cpuset=2,3`, потолок 1.5 CPU, 2.5 ГиБ RAM
Kubernetes API	`127.0.0.1:6550`, наружу не публикуется
Ingress	Traefik (встроен в k3s); ServiceLB занимает 80/443 только внутри контейнера узла
Хранилище	`local-path`, данные томов в `/opt/rb-sem2/k3d/storage` на диске хоста

2. Создание кластера (Ansible-роль `k3d`)

Платформа описана кодом: в Ansible-проект ЛР5 добавлена роль `labs/lab5/ansible/roles/k3d`. Роль делает следующее:

1. скачивает k3d и kubectl в `/opt/rb-sem2/bin` с проверкой sha256. Хэш k3d закреплён в переменных роли: в `checksums.txt` релиза путь записан как `_dist/k3d-linux-amd64`, и модуль `get_url` не находит его по имени файла;
2. создаёт кластер, если его ещё нет:

```
k3d cluster create rb --servers 1 --agents 0 --no-lb \
  --image rancher/k3s:v1.36.6-k3s1 --network rb-edge \
  --api-port 127.0.0.1:6550 --servers-memory 2560m \
```

```
  --volume /opt/rb-sem2/k3d/storage:/var/lib/rancher/k3s/storage@server:0 \
  --kubeconfig-update-default=false --kubeconfig-switch-context=false --wait
```

Узел подключается к общей docker-сети `rb-edge`, где его видит edge-nginx. `--servers-memory` выставляет docker-лимит памяти и подменяет `/proc/meminfo`, чтобы kubelet знал реальный объём памяти;

3. ограничивает CPU узла: `docker update --cpuset-cpus 2,3 --cpus 1.5`, только если текущие значения отличаются;
4. пишет kubeconfig в `/opt/rb-sem2/k3d/kubeconfig` (`0600`), а не в `~/.kube`, чтобы не трогать окружение root на общем сервере;
5. один раз генерирует секреты в `/opt/rb-sem2/k8s/secrets.env` (`0600`) и создаёт из них Secret `rb-secrets` в пространстве `rb`.

```
$ ansible-playbook site.yml --tags k3d
TASK [k3d : Бинарник k3d (проверка закреплённого sha256)] .......... changed
TASK [k3d : Бинарник kubectl (проверка sha256)] ..................... changed
TASK [k3d : Создать кластер (1 узел, сеть rb-edge, без публикации 80/443 на хост)]
changed
TASK [k3d : Ограничить CPU узла — cpuset и потолок ядер 1.5] ........ changed
TASK [k3d : Kubeconfig вне ~/.kube] ................................. changed
TASK [k3d : Секреты кластера (генерируются один раз, в git не попадают)] changed
TASK [k3d : Namespace и Secret rb-secrets] .......................... changed
TASK [k3d : Узел кластера]
  k3d-rb-server-0  Ready  control-plane  9s  v1.36.6+k3s1  K3s v1.36.6+k3s1  6.8.0-
147-generic (amd64)  containerd://2.3.7-k3s1.36
PLAY RECAP
rb-server : ok=18  changed=7  unreachable=0  failed=0
```

Через полминуты в кластере запустились системные компоненты k3s: CoreDNS, Traefik со svclb, metrics-server и local-path-provisioner. Узел с ними занял 653 МиБ из 2.5 ГиБ.

3. Влияние на сервер: проверка «до/после»

Узел k3d — привилегированный контейнер. Поэтому до и после создания кластера снимались слушающие порты, правила iptables, загруженные модули ядра и значения `sysctl` хоста. Сравнение:

Что	Изменение	Оценка
слушающие порты	+ `127.0.0.1:6550` (Kubernetes API)	только loopback, наружу ничего не открылось
docker-сети	без изменений (узел в существующей `rb-edge`)	—

Что	Изменение	Оценка
правила iptables хоста	filter 103 → 104, nat 19 → 20, raw 38 → 40	служебные правила Docker для нового контейнера
модули ядра	`br_netfilter`, `vxlan`, `xt_physdev`, `xt_comment`, `nfnetlink_log` и др.	нужны сети кластера (flannel VXLAN, kube-proxy)
sysctl хоста	`kernel.panic` 0 → 10, `kernel.panic_on_oops` 0 → 1, `vm.overcommit_memory` 0 → 1, `net.bridge.bridge-nf-call-iptables` → 1	**глобальные параметры поменял kubelet**

Kubelet при старте выставляет «рекомендуемые» для узла Kubernetes параметры ядра. Изнутри привилегированного контейнера они применяются ко всему серверу. На общем сервере это неприемлемо: например, с `panic_on_oops=1` любой kernel oops перезагрузил бы сервер вместе с соседним проектом. По согласованию с владельцем сервера исходные значения возвращены. Восстановление включено в роль `k3d`, работает только в runtime через `sysctl -w` и не создаёт файлов в `/etc`:

```
# roles/k3d/defaults/main.yml
k3d_host_sysctl_restore:
  kernel.panic: "0"
  kernel.panic_on_oops: "0"
  vm.overcommit_memory: "0"
  net.bridge.bridge-nf-call-iptables: "0"
  net.bridge.bridge-nf-call-ip6tables: "0"
  net.bridge.bridge-nf-call-arptables: "0"
```

```
host:                 kernel.panic = 0, kernel.panic_on_oops = 0,
vm.overcommit_memory = 0, bridge-nf-call-iptables = 0
inside k3d node netns: bridge-nf-call-iptables = 1
```

Кластер от этого не пострадал. В ядре 6.8 параметры `bridge-nf-call-*` действуют на каждое сетевое пространство имён отдельно, и внутри узла k3d мосты CNI по-прежнему фильтруются (значение 1). Kubelet меняет параметры только при своём запуске, поэтому после перезагрузки сервера роль `k3d` нужно прогнать повторно (`ansible-playbook site.yml --tags k3d`). Это указано в комментарии к переменным роли. Статусы контейнеров соседнего проекта до и после не изменились.

4. Манифесты

Манифесты написаны на kustomize: `k8s/base` содержит само приложение, `k8s/overlays/prod` задаёт реестр и тег образов.

```
k8s/
├── base/
│   ├── kustomization.yaml      # namespace rb, общие метки, список ресурсов
│   ├── namespace.yaml          # Pod Security Admission: enforce=baseline,
audit=restricted
│   ├── config.yaml             # ConfigMap rb-config, SQL создания 5 БД (database-
per-service)
│   ├── postgres.yaml           # StatefulSet + headless Service + PVC 2Gi
│   ├── rabbitmq.yaml           # StatefulSet + Service + PVC 1Gi
│   ├── {auth,catalog,reservation,review,notification}-service.yaml, gateway.yaml  #
Deployment + Service
│   ├── ingress.yaml            # Ingress (Traefik) + PodDisruptionBudget gateway
│   ├── networkpolicy.yaml      # default-deny + явные разрешения
│   └── backup.yaml             # CronJob pg_dump + PVC 1Gi
└── overlays/prod/kustomization.yaml   # images: rb-* -> ghcr.io/cicada-pops/rb-*
```

Ресурсы по Д37: 6 Deployment, 2 StatefulSet, 8 Service, Ingress, PDB, 4 NetworkPolicy, CronJob, 2 ConfigMap и 3 PVC. Проверка `kubeconform -strict` по схемам Kubernetes 1.36 дала 27 из 27 валидных ресурсов.

Типовой Deployment сервиса (сокращённо):

```
spec:
  replicas: 1
  strategy: { type: RollingUpdate, rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
}
  template:
    metadata:
      annotations: { prometheus.io/scrape: "true", prometheus.io/port: "8083" }
    spec:
      securityContext: { runAsNonRoot: true, runAsUser: 1000, seccompProfile: {
type: RuntimeDefault } }
      containers:
        - name: app
          image: rb-reservation-service          # kustomize: ghcr.io/cicada-
pops/rb-reservation-service:<sha>
          env:
            - name: POSTGRES_PASSWORD
              valueFrom: { secretKeyRef: { name: rb-secrets, key: POSTGRES_PASSWORD
} }
            - name: DATABASE_URL                 # подстановка $(VAR) из переменной
выше
              value: postgres://rb:$(POSTGRES_PASSWORD)@postgres:5432/reservations
            - { name: CATALOG_URL, value: "http://catalog-service:8082" }
          resources:
            requests: { cpu: 50m, memory: 96Mi }
            limits:   { cpu: 500m, memory: 192Mi }
          startupProbe:   { httpGet: { path: /health, port: http }, periodSeconds:
```

```
2, failureThreshold: 60 }
          readinessProbe: { httpGet: { path: /health, port: http }, periodSeconds:
10 }
          livenessProbe:  { httpGet: { path: /health, port: http }, periodSeconds:
20, failureThreshold: 3 }
          lifecycle: { preStop: { sleep: { seconds: 5 } } }
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: [ALL] }
          volumeMounts: [{ name: tmp, mountPath: /tmp }]
```

Особенности:

- **Конфигурация — через переменные окружения**, как и в compose. Отличие одно: задан `DATABASE_URL`, поэтому сервисы работают с PostgreSQL (поддержка добавлена в ЛР4). Схему и сиды создаёт одна реплика под `pg_advisory_lock`;
- **PostgreSQL** — StatefulSet с `volumeClaimTemplates`. Пять баз (`auth`, `catalog`, `reservations`, `reviews`, `notifications`) создаёт init-скрипт из ConfigMap при первом запуске. Сервисы обращаются к headless-сервису `postgres`;
- **RabbitMQ** — StatefulSet со стабильным именем пода (`rabbitmq-0`), поэтому имя узла Erlang и данные mnesia на PVC сохраняются при перезапуске;
- **Ingress** направляет всё на gateway, gateway маршрутизирует `/api/v1/*` к сервисам, как в compose;
- **NetworkPolicy:** входящий трафик в `rb` по умолчанию запрещён, разрешён внутри пространства, от Ingress-контроллера (`kube-system`) к gateway и от мониторинга (`monitoring`, ЛР8) к метрикам;
- **бэкап:** CronJob `postgres-backup` в 02:30 UTC делает `pg_dump -Fc` каждой базы в PVC и хранит 7 копий.

5. Маршрут снаружи

Edge-nginx из ЛР4 получил файл `conf.d/k8s.conf`. Префикс `/k8s/` срезается, и запрос уходит на Traefik внутри узла по сети `rb-edge`:

```
location = /k8s/metrics { return 404; }
location /k8s/ {
  set $k8s http://k3d-rb-server-0:80;
  rewrite ^/k8s/(.*)$ /$1 break;
  proxy_pass $k8s;
}
```

Compose-стенд из ЛР4 продолжает работать на `/`, кластер доступен на `/k8s/`, порт по-прежнему один — 8088.

6. Развёртывание

Релиз выкатывает тот же скрипт `deploy.sh`, что и в ДЗ6, в режиме `k8s <sha>`. Скрипт:

1. проверяет, что узел запущен и Secret `rb-secrets` существует;
2. загружает образы `ghcr.io/cicada-pops/rb-*:<sha>` (из GHCR или уже скачанные на сервер) и импортирует их прямо в containerd узла: `docker save … | docker exec -i k3d-rb-server-0 ctr -n k8s.io images import -`. Кластеру не нужны ни доступ к реестру, ни pull-секреты;
3. копирует манифесты нужного коммита во временный каталог, добавляет слой `overlays/release` с `images: newTag: <sha>` и выполняет `kubectl diff` и `kubectl apply -k`;
4. ждёт `rollout status` всех Deployment и проверяет, что через Ingress отвечает именно новая версия;
5. при сбое выполняет `kubectl rollout undo` для всех Deployment.

Первое развёртывание выполнено вручную на сервере. Образы этого коммита уже были скачаны конвейером ДЗ6:

```
$ echo "" | /opt/rb-sem2/bin/deploy.sh k8s 962856ca8885c8c7e981887e5bafde28bb985988
[17:47:07] [k8s 962856c] sources at 962856c feat(lab7): add k8s manifests and k3d
platform
[17:47:23] [k8s 962856c] images imported into k3d-rb-server-0 containerd (6)
namespace/rb configured
configmap/postgres-init created
...
statefulset.apps/postgres created
statefulset.apps/rabbitmq created
cronjob.batch/postgres-backup created
ingress.networking.k8s.io/restaurant-booking created
networkpolicy.networking.k8s.io/default-deny-ingress created
deployment "gateway" successfully rolled out
...
deployment "notification-service" successfully rolled out
[17:49:48] [k8s 962856c] k8s release 962856ca8885c8c7e981887e5bafde28bb985988 is
live
NAME                                  READY   UP-TO-DATE   AVAILABLE
deployment.apps/auth-service          1/1     1            1
deployment.apps/catalog-service       1/1     1            1
deployment.apps/gateway               2/2     2            2
deployment.apps/notification-service  1/1     1            1
deployment.apps/reservation-service   1/1     1            1
deployment.apps/review-service        1/1     1            1
statefulset.apps/postgres             1/1
statefulset.apps/rabbitmq             1/1
real    2m41s
```

Следующие релизы выкатываются автоматически конвейером ДЗ8.

7. Проверка

Снаружи (с рабочего компьютера, через публичный порт 8088):

```
$ curl -s http://<SERVER_IP>:8088/k8s/health
{"status":"ok","service":"api-
gateway","version":"962856ca8885c8c7e981887e5bafde28bb985988"}
register -> token 192 chars
restaurants (Москва): 3 [('La Piazza', 4.5), ('Тбилисо', 5), ('Самовар', 0)]
reservation: 1 confirmed Sakura
notifications: ['Бронирование подтверждено']
swagger /k8s/api/docs/ -> 200
/k8s/metrics -> 404
```

Пройдена вся цепочка: регистрация в auth-service (PostgreSQL), каталог с рейтингами (catalog → review), бронь (reservation → catalog), событие через RabbitMQ и уведомление от notification-service.

Внутри кластера:

```
== PVC
data-postgres-0   Bound  2Gi  local-path
data-rabbitmq-0   Bound  1Gi  local-path
postgres-backups  Bound  1Gi  local-path      # привязывается при первом запуске
бэкапа (WaitForFirstConsumer)
== NetworkPolicy (пробные поды busybox)
из пространства np-probe -> postgres:5432 (TCP):  BLOCKED
из пространства np-probe -> catalog-service:      BLOCKED
из пространства rb       -> catalog-service:      ALLOWED
== Persistence (удаление пода postgres-0)
reservations before: 1
reservations after pod re-creation: 1
== Backup (ручной запуск задания из CronJob)
40.0K /backups/20261110T180930Z
auth.dump 3628  catalog.dump 17791  notifications.dump 3813  reservations.dump 3485
reviews.dump 3518
== Resources
k3d-rb-server-0   CPU 206m   MEMORY 1579Mi (60%)
docker: cpuset=2,3 cpus=1500000000 memory=2684354560
```

Потребление подов в простое (`kubectl top`): сервисы по 19–31 МиБ, PostgreSQL 56 МиБ, RabbitMQ 126 МиБ. Узел целиком (k3s, системные поды, приложение) — 1.56 ГиБ из 2.5 ГиБ.

8. Найденная проблема: холодный старт

При первом развёртывании `reservation-service` и `review-service` один раз перезапустились по `startupProbe`:

```
lastState: {"terminated": {"exitCode": 137, "startedAt": "17:47:26", "finishedAt":
"17:49:44"}}
[db] connect 9/30 failed (getaddrinfo ENOTFOUND postgres), retry in 2s
Container app failed startup probe, will be restarted
```

Узел в это время скачивал образы PostgreSQL и RabbitMQ, и зависимости поднялись позже обычного. Сервисы-издатели по очереди ждали БД (до 60 с) и брокер (до 60 с) и только потом открывали порт, поэтому 120 секунд startup-пробы не хватило. При этом публикация событий у нас best-effort (ДЗ5): при недоступном брокере ответ клиенту не ломается. Поэтому подключение к RabbitMQ перенесено в фон (`void initBus()`), и старт ждёт только базу. Исправление выкачено конвейером ДЗ8, и в следующих релизах рестартов при старте нет.

Вывод

Микросервисное приложение развёрнуто в Kubernetes на облачной ВМ по техническому дизайну ДЗ7. Платформа (k3s через k3d) описана Ansible-ролью: бинарники ставятся с проверкой контрольных сумм, кластер изолирован в контейнере и ограничен 1.5 CPU и 2.5 ГиБ, API доступен только локально, секреты создаются на сервере. Приложение описано манифестами kustomize: stateless-сервисы с пробами, лимитами и жёстким securityContext, PostgreSQL и RabbitMQ как StatefulSet с постоянными томами, Ingress, сетевые политики по принципу «запрещено всё, кроме нужного», PDB и ежедневный бэкап. Проверены сквозной сценарий снаружи, изоляция сети, сохранность данных при пересоздании пода БД и бэкап. Отдельно проконтролировано влияние на общий сервер: обнаружено, что kubelet меняет глобальные параметры ядра хоста, и они возвращены в исходное состояние, а восстановление автоматизировано. Найденная при холодном старте проблема со startup-пробой устранена в коде приложения.